

FDND CMS Migration Guideline

Yaroslav Kazeev, Jun 25, 2026

Scope	3
Data Modeling	3
Customer's vision	3
Presence of Errors and Bad Practices	3
Data Migration	4
Planning	4
Conventions	4
Implementation	5
Manual Migration	5
Migration Scripts	6
Acknowledgements	10

Scope

This guideline is intended for use by FDND students who follow the FDND curriculum, practices, and tech stack and are working on the projects listed here: <https://dashboard.fdnd.nl/>. This implies at least some familiarity with SvelteKit and CMSes that use GraphQL, such as Hygraph and Directus.

This guideline captures the experience gained by familiarizing oneself with the codebases at <https://github.com/fdnd-agency/biebinbloei.nl> and <https://github.com/fdnd-agency/visual-thinking>, as well as the stakeholders' requirements for those projects.

The actual Hygraph-Directus migration happened only to the 'Visual Thinking' project. The conclusions and practices used here may be suboptimal in other cases.

Data Modeling

Customer's vision

The customer requirements should be set as early as possible. Chances are that the content in the CMS is outdated, or the customer wants to change the website's look. Then migrating the content as-is makes little to no sense.

Some data that exists in Hygraph may reflect experimental features. For example, 'likes' and 'comments' are omitted in the current "Visual Thinking" website. Such features pertain mostly to a social platform, rather than an educational one. Comprehensive feedback can be provided via email rather than via two-word comments or 'likes'.

The website customer should clarify their vision for the platform, which will determine whether these features need to be migrated from Hygraph to Directus.

Presence of Errors and Bad Practices

This is another reason not to clone the existing Hygraph model directly in Directus. The project may have inconsistencies both in the previous Hygraph model and the code, which are not desirable to replicate, such as: `'.\biebinbloei.nl\src\routes\geveltuin\+page.svelte'`

```
<picture>
  <img src={data.facadeGardenActions[0].image.url} alt="" />

  <p class="resident-henk">
    {data.facadeGardenActions[1].description}
  </p>
</picture>
```

Here, the indexes are used to cherry-pick from the array retrieved from the CMS. This necessitates changing the code when a new entry is added to the CMS, rather than managing content solely on the CMS side.

Both Hygraph and Directus CMS can hide array elements from being retrieved in query responses by assigning the correct status to them. Hygraph has this ability out of the box, so elements with status “published” are retrievable; elements with status “draft” are not.

Directus requires the appropriate setup to get a similar behavior:

1. Go to Settings → Access Policies
2. Modify the existing 'Public Policy'
3. Add Permission to a given collection: Click Read → Use Custom → Item Permissions → Add Filter: status = published

Note: The Public Policy was not modified during the ‘Visual Thinking’ migration project; all statuses are retrieved from Directus indiscriminately. For example, this results in all pictures (visuals within each drawing method step) being shown on the web page instead of retrieving the one at index 0. All current Items in the project have ‘Draft’ status or lack the status field, and need an upgrade to make the policy work.

Errors in the code and features that should not be migrated could be further identified during the next phase.

Data Migration

Planning

This step could be done using the ‘divide and conquer’ principle. Navigating between the pages usually triggers the execution of a particular GraphQL query. Each team member is responsible for creating a suitable tree-like model in Directus, with the caveats discussed earlier.

Hygraph schema types are equivalent to Directus collections. If such collections are used in more than one query, this should be identified as early as possible to ensure compliance. In case of the ‘Visual Thinking’ project, the “Categorie” was used in Hygraph both to show content on the “Kennisclips” page as well as a part of the query in the “Tekenmethod stappenplan”.

Later, two collections were created in Directus at the customer’s request to facilitate adding and deleting items from ‘Categories’ and ‘Kennisclips’ independently.

Conventions

Those naming conventions in the “Visual Thinking” data migration project are to supplement and change the existing FDND ones for better consistency, especially across Frontend-CMS features:

1. Dutch is, if possible, the default principal language for UI content, collections, and fields in Directus CMS and GraphQL queries; unless the site is multilingual, English is preferred. The identical names are preferred.
2. JS/Sweltekit code should use English unless properties are imported from the CMS without modification.

3. Abbreviated versions of project names should be preferred for compound collection names in Directus CMS.
4. Collection and field names should have a lowercase snake case format

Implementation

Manual Migration

Most of the items were migrated by copying and pasting from Hygraph to Directus manually; this was chosen because of three factors:

1. The success of the migration is verified by a visual comparison between the older and newer versions of the given page in any case, which makes manual migration more gradual and controllable.
2. Each nesting level in GraphQL queries is a potential source of bugs, making it impractical to create and debug migration scripts for a small number of items that pertain to a specific element within the data tree branch.
3. There is a risk of corrupting the data in Directus because all FDND projects share the same instance.

This table shows the number of items transferred in Directus; all were transferred by hand except for those migrated by a script: the vt_tekenmethodes (2 text fields in 61 items) and 14 PDF files in the vt_tekenmethodes_files collection.

Qty	Collection
1	vt_aboutpage
6	vt_artikel
1	vt_artikelen_page
23	vt_bronnen
6	vt_categorieen
1	vt_homepage
6	vt_kennisclips
1	vt_kennisclips_page
4	vt_minicursussen
1	vt_minicursussen_page
28	vt_minicursussen_slides
1	vt_over_page

61	vt_tekenmethodes
44	vt_tekenmethodes_files
9	vt_tekenmethodes_materialen
1	vt_tekenmethodes_page
272	vt_tekenmethodes_stappen
93	vt_tekenmethodes_stappen_files
43	vt_tekenmethodes_vt_bronnen
118	vt_tekenmethodes_vt_categorieen
177	vt_tekenmethodes_vt_tekenmethodes_materialen
897	TOTAL

For the 'VisualThinking' project, using scripts offered no apparent advantage over manually migrating a small dataset, which our team of five people performed.

Migration Scripts

The JavaScript script below (`.\visual-thinking\migration\check_missing_tekenmethods_graphql.js`) illustrates a typical example of migrating files from Hygraph to Directus while maintaining proper relationships.

Notes:

- **Execution:** Run the script using the following CLI command: `node --env-file=.env migration\check_missing_tekenmethods_graphql.js`
- **Prerequisites:** The script requires that the `tekenmethods` entries have already been created in Directus and have, at minimum, their `slug` fields populated.
- **API Choice (GraphQL vs REST):** Initially, the Directus REST API was used alongside a session token (`DIRECTUS_KEY`). Switching to GraphQL allows for more succinct code and avoids granting excessive public API permissions to Directus collections.
- **Directus Static Token:** To create a static token, navigate to the Directus dashboard at <https://fdnd-agency-production.directus.app/admin/users>. Click a user, then in the "Admin Options" section, click the + button to generate a new token.
- **Batch Updating:** This script supports batch updating an entire array of `tekenmethods`. As a safe strategy, it's recommended to test the update process on a single item first before running the full batch.
- **ID Resolution:** Retrieving the `tekenMethodeDirectusId` is a necessary step. Because both Hygraph and Directus use PostgreSQL under the hood, relational updates must be

done via primary IDs rather than slugs. Specifically, calling `updateItem("vt_tekenmethodes", tekenMethodeDirectusId, { pdf: pdfId })` inserts a new entry into the junction table (`vt_tekenmethodes_files`), linking `vt_tekenmethodes_id` to `directus_files_id`.

```
//
import {
  createDirectus,
  graphql,
  staticToken,
  updateItem,
  uploadFiles,
} from "@directus/sdk";
import { GraphQLClient } from "graphql-request";

const directusUrl = process.env.DIRECTUS_URL;
const directusToken = process.env.DIRECTUS_TOKEN; // Ensure DIRECTUS_TOKEN is
available

const directus = createDirectus(directusUrl)
  .with(staticToken(directusToken))
  .with(graphql());

const hygraph = new GraphQLClient(process.env.HYGRAPH_URL, {
  headers: {
    Authorization: `Bearer ${process.env.HYGRAPH_KEY}`,
  },
});

const query = `
  query Methods {
    methods {
      slug
      pdf {
        url
      }
    }
  }
`;

const Idquery = `
  query Tekenmethodes($slug: String!) {
    vt_tekenmethodes(filter: { slug: { _eq: $slug } }) {
```

```

        id
    }
}
`;

async function migratePdfs() {
    try {
        const response = await hygraph.request(query);
        const methods = response.methods; // array of methods.
        const methodsWithPdfs = methods.filter((method) => method.pdf !== null);

        console.log(
            `Found ${methodsWithPdfs.length} methods with PDFs. Starting migration...`
        );

        for (const methodWithPdf of methodsWithPdfs) {
            try {
                console.log(`Processing PDF for slug: ${methodWithPdf.slug}`);

                // 1. Fetch PDF from Hygraph
                const pdfResponse = await fetch(`${methodWithPdf.pdf.url}`);
                if (!pdfResponse.ok) {
                    throw new Error(
                        `Failed to fetch PDF for ${methodWithPdf.slug}:`
                        `${pdfResponse.statusText}`
                    );
                }
                const pdfBlob = await pdfResponse.blob();

                // 2. Upload PDF to Directus
                const formData = new FormData();
                formData.append("file", pdfBlob, `${methodWithPdf.slug}.pdf`);
                const postPdfResponse = await directus.request(uploadFiles(formData));
                const pdfId = postPdfResponse.id;

                // 3. Find associated Tekenmethode in Directus
                const idResponse = await directus.query(Idquery, {
                    slug: methodWithPdf.slug,
                });
                if (

```



```

        !idResponse.vt_tekenmethodes ||
        idResponse.vt_tekenmethodes.length === 0
    ) {
        console.warn(
            `[Warning] No vt_tekenmethodes found for slug:
${methodWithPdf.slug}. Skipping update.`
        );
        continue;
    }
    const tekenMethodeDirectusId = idResponse.vt_tekenmethodes[0].id;

    // 4. Update Tekenmethode with PDF ID
    await directus.request(
        updateItem("vt_tekenmethodes", tekenMethodeDirectusId, { pdf: pdfId
    })
    );

    console.log(`Successfully migrated PDF for ${methodWithPdf.slug}`);
} catch (err) {
    console.error(`Error processing ${methodWithPdf.slug}:`, err);
}

}

console.log("Migration completed.");
} catch (err) {
    console.error("Migration failed:", err);
}

}

migratePdfs();
//

```

There are more scripts in the `.\visual-thinking\migration` folder; one can use them as a basis for creating new ones.

Acknowledgements

Our joint student group of HvA and HYF students consisted of:

- Bilal Budil
- Chaim Cadeau
- Karim Hanna
- Quinten Vos

- Yaroslav Kazeev

Migration projects were part of the internship in Block 4 (13/04 - 03/07 2026) of the HvA-FDND curriculum.

Mariska Gunsing and Charley Muhren are customers and owners of the biebinbloei.nl and visualthinking.school websites respectively.

Joost Faber, the teacher at FDND and the product owner for both projects, organized this joint internship.

I am very grateful to learn something new and to the people mentioned here for their positive collaboration and organization.

Yaroslav Kazeev, the HYF student